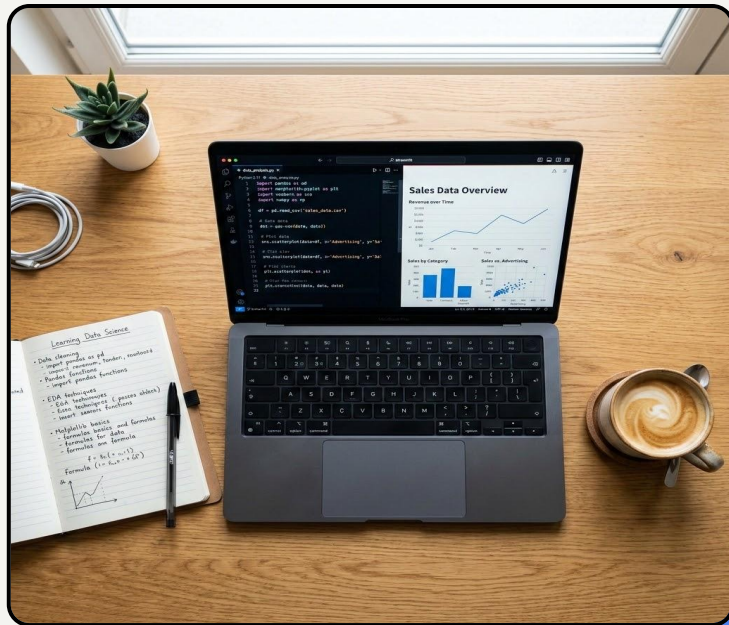




Aula 1: Introdução à Ciência de Dados com Python e Pandas



**Prof. Dr. Danilo César
Pereira**

Objetivos da Aula

◆ Compreender os fundamentos do Python para análise de dados.

◆ Aprender a manipular dados com a biblioteca Pandas.

◆ Criar as primeiras visualizações gráficas.

◆ Iniciar a exploração de datasets reais.

Python como Calculadora

O Python funciona como uma calculadora poderosa e flexível para cientistas de dados.

O ecossistema suporta de forma nativa e eficiente tanto operações tradicionais quanto cálculos de alta performance:

- **Aritmética Básica:** Adição (+), subtração (-), multiplicação (*) e divisão (/).
- **Operações Avançadas:** Exponenciação (**) e divisão inteira (//).

Compreender essas operações é o primeiro passo essencial para realizar cálculos estatísticos e manipulações complexas em grandes volumes de dados.

```
# Operações aritméticas básicas e avançadas
soma = 10 + 5 # Adição: 15
mult = 4 * 3 # Multiplicação: 12
expo = 2 ** 3 # Exponenciação: 8
div_int = 11 // 3 # Divisão Inteira: 3
```

Terminal de Saída

```
>>> print(soma, mult, expo, div_int)
15 12 8 3
```

Bibliotecas no Python

As bibliotecas são pacotes de código pré-escrito que expandem as funcionalidades nativas do Python. Na Ciência de Dados, esse ecossistema é incrivelmente vasto.

Importamos essas ferramentas prontas usando o comando `import`, o que evita a necessidade de reinventar a roda a cada novo projeto desenvolvido.



NumPy

- ◆ Cálculos numéricos de alta performance
- ◆ Processamento eficiente de arrays e matrizes

Pandas

- ◆ Manipulação de grandes tabelas de dados
- ◆ Estruturas flexíveis como DataFrames

Matplotlib

- ◆ Criação de gráficos bidimensionais completos
- ◆ Visualizações de dados customizadas

Estrutura de Dados: DataFrame

O que é um DataFrame?

A estrutura central da biblioteca Pandas é o **DataFrame**. Ele representa uma tabela de dados bidimensional, organizada de forma clara e estruturada.

Tabela Bidimensional: Estrutura com linhas e colunas flexíveis.

Familiaridade: Muito semelhante a uma planilha do Excel ou uma tabela SQL.

Construção Dinâmica: Criado a partir de um dicionário Python com a função `pd.DataFrame()`.

Criação com dicionário de dados

```
import pandas as pd
dados = {
    'Produto': ['A', 'B'],
    'Vendas': [100, 250]
}
df = pd.DataFrame(dados)
```

Saída do DataFrame (Terminal):

Produto Vendas

```
0 A 100
1 B 250
```

| Exploração Inicial de DataFrames

Após criar ou carregar um DataFrame, o Pandas oferece métodos essenciais para a exploração inicial:

1 .head()

Exploração

Exibe as primeiras cinco linhas, permitindo uma rápida inspeção visual.

Objetivo

Validação imediata do layout, nomes de colunas e alinhamento dos dados carregados.

2 .info()

Exploração

Fornece um resumo técnico, incluindo os tipos de dados de cada coluna e a contagem de valores não nulos.

Objetivo

Identificação de consistência estrutural e detecção preliminar de dados ausentes (null values).

3 .describe()

Exploração

Gera estatísticas descritivas automáticas, como média, desvio padrão, valores mínimos e máximos.

Objetivo

Análise da distribuição estatística de todas as colunas de dados puramente numéricos.

Seleção de Colunas no Pandas

Em análises reais, frequentemente precisamos focar em variáveis específicas. O Pandas permite selecionar colunas de forma simples, alterando a estrutura final de retorno.

◆ Seleção de Coluna Única (Series)

Retorna uma estrutura de dados unidimensional. Utiliza-se colchetes e o nome da coluna em formato de string.

Exemplo: `df['nome_da_coluna']`

◆ Seleção de Múltiplas Colunas (DataFrame)

Retorna um novo DataFrame menor e mais focado. Passa-se uma lista de nomes de colunas simultaneamente.

Exemplo: `df[['coluna1', 'coluna2']]`

```
# Selecionando uma coluna (retorna Series)  
serie_idades = df['idade']
```

```
# Selecionando múltiplas colunas (retorna DataFrame)
```

```
sub_df = df[['nome', 'idade']]
```

```
>> type(serie_idades)  
pandas.core.series.Series (Unidimensional)
```

```
>> type(sub_df)  
pandas.core.frame.DataFrame (Multidimensional)
```

Filtragem e Indexação Booleana no Pandas

Isolando dados de forma eficiente

A filtragem de registros permite isolar linhas que atendem a critérios específicos. No Pandas, utilizamos a **indexação booleana** para alcançar esse objetivo com alta performance.

◆ **Como funciona:** O interpretador avalia cada linha e retorna apenas aquelas onde o valor da expressão lógica é estritamente **True**.

◆ **Filtros Complexos:** Combine múltiplas condições usando os operadores lógicos & (E) e | (OU).

```
import pandas as pd
```

```
# 1. Filtragem Simples (Idade > 18)
```

```
df_maiores = df[df['idade'] > 18]
```

```
# 2. Filtragem Composta com Operador '&'
```

```
condicao = (df['idade'] > 18) & (df['cidade'] == 'SP')
```

```
df_filtrado = df[condicao]
```



Dica Importante:

Sempre utilize parênteses ao combinar múltiplos critérios lógicos. A falta de parênteses em condições compostas gera erros de sintaxe ou avaliações inesperadas pelo interpretador do Pandas.

Importando Dados com Pandas

Para trabalhar com dados reais, utilizamos a função **pd.read_csv()** do Pandas, que importa arquivos de texto separados por vírgula diretamente para um DataFrame.

Como exemplo prático, importaremos um dataset público do IMDB contendo informações sobre milhares de filmes. O processo converte instantaneamente o arquivo bruto em uma estrutura tabular manipulável na memória, pronta para a aplicação de métodos de limpeza e análise estatística.

```
# Importar biblioteca e carregar dataset
import pandas as pd
url = "https://tinyurl.com/imdb-movies-dataset"
df = pd.read_csv(url)
df.head()
```



Dataset de Filmes do IMDB

A estruturação tabular permite indexar instantaneamente títulos, diretores, notas, gêneros e faturamento de milhares de obras cinematográficas armazenadas na nuvem.

Exploração Inicial & Estatísticas do IMDB

Atributos Principais I

A exploração do dataset revela colunas fundamentais que identificam cada obra:

Título do Filme: A identidade única da obra no catálogo do IMDB.

Diretor: Permite analisar a autoria, estilo cinematográfico e o impacto das direções na crítica.

Atributos Principais II

Outros metadados estruturam a contextualização temporal e de mercado:

Ano de Lançamento: Essencial para mapear a evolução histórica das produções.

Gênero: Classificação temática que agrupa as produções (Drama, Ação, Sci-Fi).

Análise Estatística

A aplicação rápida do método `.describe()` extrai insights numéricos imediatos sobre as avaliações:

Nota Média Histórica: **Métrica central** das avaliações de todos os filmes.

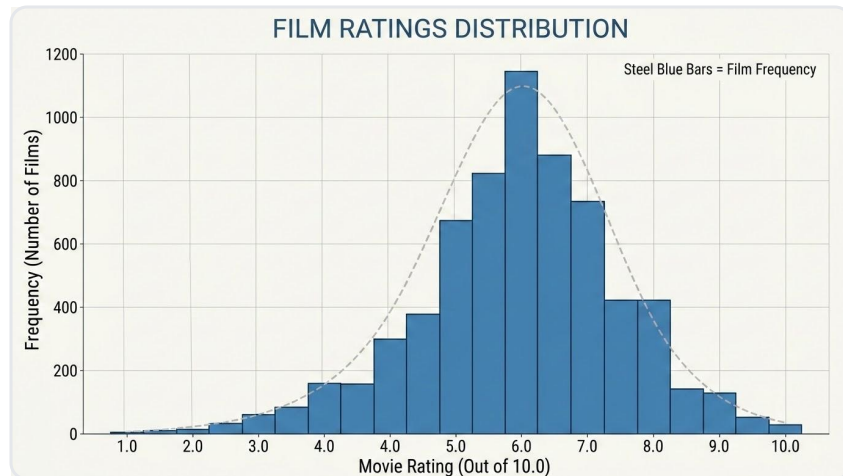
Ano Mais Antigo: Identificação do **marco inicial** de registro do dataset.

Dispersão: Entendimento do desvio padrão das notas dadas pelo público.

Visualização de Dados: Distribuição de Notas do IMDB

```
import pandas as pd  
df['nota'].plot(kind='hist', bins=20)
```

- ◆ **Distribuição Normal:** Permite identificar imediatamente se as avaliações seguem uma distribuição simétrica (curva de sino).
- ◆ **Assimetria das Notas:** Revela tendências sistemáticas dos usuários de atribuírem notas consistentemente mais altas ou mais baixas.
- ◆ **Concentração:** Localiza o intervalo exato de notas onde se concentra o maior volume de avaliações da base de dados.



O histograma facilita a identificação rápida da frequência de cada faixa de nota, permitindo analisar visualmente o comportamento das avaliações do conjunto.



A visualização de dados facilita a compreensão de padrões complexos em grandes conjuntos de dados.

Exportando Dados com Pandas

Após limpar, filtrar e analisar os dados, é fundamental salvar os resultados. O Pandas oferece o método `.to_csv()` para exportar o DataFrame manipulado de volta para um arquivo no disco.

Boas Práticas de Organização:

- **Arquivo Separado:** Recomenda-se salvar os dados processados em um novo arquivo, separado do arquivo original (raw data).
- **Preservação:** Evite sobrescrever a base original para prevenir perdas acidentais de informações brutas.

```
# 1. Importar a biblioteca
import pandas as pd

# 2. Exportar o DataFrame para um novo CSV
df.to_csv('dados_processados.csv', index=False)

# O arquivo bruto original permanece intocado.
```



DICA DE EXPORTAÇÃO: Usar o parâmetro `index=False` evita que o índice numérico padrão do DataFrame seja salvo como uma coluna adicional desnecessária no arquivo exportado.

Atividade Prática: Explorando o Kaggle

+50.000

Datasets Públicos

A maior plataforma global para a comunidade de Ciência de Dados e Machine Learning.

O Kaggle hospeda milhares de repositórios de alta qualidade, cobrindo tópicos que vão desde finanças e saúde até esportes e cultura pop.

Na atividade de hoje, vocês explorarão este acervo para encontrar um dataset que desperte seu **interesse pessoal**.

O grande objetivo é aplicar de forma prática as técnicas de importação e exploração do **Pandas** em um contexto que seja verdadeiramente motivador para você.



INÍCIO DA ATIVIDADE PRÁTICA — TEMPO ESTIMADO: 15 MINUTOS

Como Navegar Eficientemente no Kaggle

Para encontrar os melhores recursos para seus projetos, acesse a aba '**Datasets**' na página inicial do Kaggle e siga os passos estruturados abaixo para garantir dados de alta qualidade.

1

Busca por Temas

Utilize a barra de pesquisa principal do Kaggle para buscar temas específicos de interesse para a sua análise.

2

Filtros e Licenças

Aplique filtros por formato de arquivo (**priorize arquivos CSV** para esta aula) e verifique atentamente as licenças de uso permitidas.

3

Usability Score

Preste atenção na pontuação de usabilidade do dataset. Pontuações mais altas indicam dados bem documentados e estruturados.



Dica de Sucesso: Datasets limpos e prontos para análise reduzem drasticamente o tempo de preparação de dados, sendo ideais para iniciantes na área focarem no aprendizado de algoritmos e visualizações.

Datasets Recomendados para Iniciantes

Para quem está em dúvida, existem datasets clássicos recomendados para iniciantes que servem de base teórica e prática.

1 Dataset Iris

Florística / Classificação

O dataset **Iris** é excelente para entender a **classificação** básica de plantas.

A análise é realizada utilizando medidas reais das **pétalas** e sépalas para agrupar três espécies distintas.

2 House Prices

Regressão / Preços

O dataset **House Prices** oferece dados reais e complexos do mercado imobiliário.

Ideal para aplicar técnicas de regressão e aprender a **prever valores** de propriedades com múltiplos fatores determinantes.

3 Play Store

Análise de Texto & Notas

Baseado em avaliações de aplicativos, sendo ideal para aplicar práticas de **análise de texto básico**.

Facilita o estudo de **correlação** entre notas dadas por usuários, volume total de downloads e categorias de apps.

Etapa Inicial: Ficha Técnica e Dicionário de Dados

Importância Analítica

Ao escolher um dataset, a primeira etapa analítica é documentar sua **ficha técnica**. Isso inclui registrar a origem dos dados, o contexto de coleta e o número total de registros e variáveis.

É crucial ler o **dicionário de dados** fornecido no Kaggle, que garante a interpretação correta durante todo o ciclo do projeto:

- ◆ Explica o significado exato de cada coluna analítica.
- ◆ Define a unidade de medida utilizada nos registros.
- ◆ Mapeia os valores possíveis para variáveis categóricas.

TEMPLATE DE DOCUMENTAÇÃO (SUGESTÃO DE USO)

Origem dos Dados: [Ex: Kaggle / Portal Transparência]

Contexto de Coleta: [Ex: Dados coletados em sensores de...]

Total de Linhas: [Registros / Amostras no dataframe]

Total de Colunas: [Variáveis explicativas e target]

Variáveis Críticas: [Identificadores e chaves primárias]

Link de Origem: [URL completa do repositório público]



Dica Prática: Replique esta estrutura de Ficha Técnica em uma célula Markdown no topo de seu Jupyter Notebook como boa prática.

Introdução ao Dataset do Titanic



O início no Machine Learning

O dataset do Titanic é um dos problemas de introdução ao Machine Learning mais famosos do mundo. Ele contém dados demográficos e de viagem dos passageiros reais do RMS Titanic.

Dados dos Passageiros

O dataset detalha informações cruciais sobre cada indivíduo a bordo:

Idade e Sexo: Perfil demográfico básico dos passageiros.

Classe e Tarifa: Indicadores socioeconômicos da cabine ocupada.

Sobrevivência: A variável alvo indicando se o passageiro sobreviveu.

Objetivo Analítico Clássico

Descobrir quais características e dados demográficos aumentavam as chances de sobrevivência durante o desastre.

Formulação de Hipóteses: Titanic Dataset

Antes de escrever código, devemos formular perguntas de negócio ou de pesquisa claras. A formulação de hipóteses guia a exploração dos dados de forma estruturada.

1 Variável: Gênero

A taxa de sobrevivência foi significativamente maior para mulheres do que para homens?

2 Variável: Classe

Passageiros da primeira classe tiveram prioridade nos botes salva-vidas em comparação com a terceira classe?

3 Variável: Idade

Crianças acompanhadas pelos pais tiveram maiores chances de sobrevivência?



Dica: Sempre estruture perguntas claras que conectem as features disponíveis no dataset com as variáveis de interesse.

Importação de Dados no Google Colab

1 Upload Direto (Temporário)

Importação rápida de arquivos locais via biblioteca `google.colab.files`.

Atenção: Os arquivos são apagados permanentemente ao fechar a sessão do notebook.

```
from google.colab import files
uploaded = files.upload()

# Após o upload, leia com o pandas:
import pandas as pd
df = pd.read_csv(next(iter(uploaded)))
```

2 Montagem do Google Drive

Integração e montagem do seu drive virtual como volume local usando a função `drive.mount()`.

Recomendado: Permite acesso persistente e seguro a todos os seus arquivos do Drive.

```
from google.colab import drive
drive.mount('/content/drive')

# Acesse arquivos diretamente no seu Drive:
import pandas as pd
path = '/content/drive/My Drive/data.csv'
df = pd.read_csv(path)
```



Dica de Desenvolvimento: Para projetos de longo prazo ou conjuntos de dados extensos, opte sempre pela montagem do Google Drive para evitar novos uploads a cada reinicialização de ambiente.

Prática: Carregando e Verificando Dados

Agora é o momento da prática!

Siga os passos para carregar e inspecionar seu arquivo CSV no ambiente de desenvolvimento.

1

Carregue o Dataset no Colab

Envie o dataset escolhido do Kaggle diretamente para o ambiente do Google Colab.

2

Crie o DataFrame do Pandas

Utilize a função `pd.read_csv()` para ler seu arquivo e iniciar a manipulação estruturada.

3

Execute a Verificação Inicial

Chame `.head()`, `.info()` e `.shape` para conferir a formatação e os delimitadores das colunas.

Execução no Google Colab

```
import pandas as pd

# 1. Carregar dataset do Kaggle

df = pd.read_csv('meu_dataset.csv')

# 2. Métodos de verificação inicial

print(df.shape)
print(df.info())
```

CONSOLE OUTPUT

```
(15000, 8)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 8 columns):
# Column Non-Null Count Dtype
0 id 15000 non-null int64
```



Resolva quaisquer erros de importação antes de prosseguir.

Análise Exploratória de Dados (EDA)

A fase essencial para compreender a "personalidade" do dataset antes da modelagem.

◆1. Valores Nulos

Foco em identificar a presença de valores nulos (dados faltantes) para avaliar a integridade e planejar o tratamento correto das lacunas.

◆2. Escala Numérica

Observar a escala dos dados numéricos para identificar discrepâncias de grandeza (por exemplo, idades variam de 0 a 100, mas salários variam de milhares a milhões).

◆3. Outliers e Anomalias

Detectar anomalias óbvias ou outliers que possam distorcer as análises estatísticas e prejudicar o desempenho dos algoritmos de aprendizado.

◆4. Próximos Passos

A EDA funciona como alicerce essencial, fundamentando todas as decisões subsequentes de limpeza, engenharia de recursos e modelagem final de dados.

Classificação de Variáveis

Para analisar dados corretamente, precisamos classificar as variáveis.

Variáveis Numéricas

Representam quantidades mensuráveis.

- **Contínuas:** valores reais (como altura e salário)
- **Discretas:** valores inteiros (como número de filhos)

No Pandas: `int64` ou `float64`

Variáveis Categóricas

Representam qualidades ou grupos.

- **Nominais:** sem ordem intrínseca (como cores ou cidades)
- **Ordinais:** com ordem hierárquica (como níveis de escolaridade)

No Pandas: geralmente `object`

Análise Univariada

A **análise univariada** foca em examinar uma única variável por vez para entender sua distribuição e características principais.

Para **variáveis numéricas**, o foco principal está em identificar a tendência central dos dados e sua respectiva variabilidade:

Média Aritmética

Representa o centro de gravidade dos dados, calculada somando todos os valores e dividindo pelo total de observações.

Mediana

O valor central que divide o conjunto de dados ordenados exatamente ao meio (50% acima, 50% abaixo), sendo robusto a outliers.

Desvio Padrão

Indica a dispersão ou afastamento médio dos dados em relação à média, apontando o grau de homogeneidade da variável.

Para **variáveis categóricas**, analisamos a distribuição das frequências para descobrir a contagem e proporção de ocorrência de cada classe.

Isso ajuda a identificar desbalanceamentos severos, como ter 90% dos registros pertencentes a uma única categoria no dataset.

```
# Gerando tabelas de frequência
df['categoria'].value_counts()
```

```
Out[1]:
Categoria_A 8120 (90.2%) Categoria_B 530 ( 5.9%) Categoria_C
350 ( 3.9%) dtype: int64
```

Alerta: Altamente Desbalanceado (90% na Categoria A)

Escolha da Visualização de Dados

Regras de Escolha de Gráficos

A escolha da visualização depende do tipo de dado:

Variáveis Numéricas: O histograma é ideal para mostrar a distribuição e a densidade dos valores ao longo de um eixo contínuo.

Variáveis Categóricas: O gráfico de barras é a escolha correta, pois exibe claramente a contagem ou proporção de cada grupo discreto.

Utilizando bibliotecas como **Matplotlib** e **Seaborn**, podemos criar essas visualizações com poucas linhas de código.

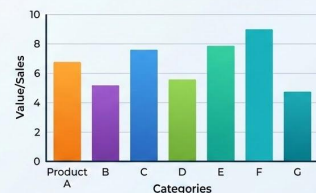
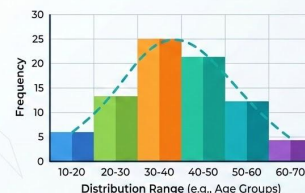
DATA VISUALIZATION COMPARISON



HISTOGRAM PLOT
(Continuous Data)



VERTICAL BAR CHART
(Categorical Data)



HISTOGRAM PLOT (Continuous Data)

- Represents frequency distribution.
- Bars are connected (no gaps).
- X-axis represents ranges (Bins).
- Shows data shape/distribution.



VERTICAL BAR CHART (Categorical Data)

- Compares discrete values across categories.
- Bars are separated (gaps).
- X-axis represents distinct groups.
- Shows differences in magnitude.

```
# Python Code Snippet
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(data=df, x="num_var") # ou sns.barplot()
```


| Documentação em Ciência de Dados

notebook_titanic.ipynb

```
In [1]: import pandas as pd
df['Survived'].value_counts(normalize=True)
```

Análise de Sobrevivência
Insight 1: A taxa média de sobrevivência geral mapeada na base foi de **38%**.
Insight 2: Mulheres e crianças do group prioritário apresentaram taxas significativamente mais elevadas.



Dica de Sucesso

Mantenha o hábito de registrar as suas conclusões imediatamente após a plotagem dos gráficos. Isso preserva o fluxo lógico da análise e poupa tempo valioso em revisões futuras.

Comunicação de Resultados

A comunicação dos resultados é uma habilidade essencial. Apresentar descobertas para a turma desenvolve a capacidade de traduzir jargões técnicos em insights.

◆ Sintetize Descobertas

Formule frases curtas, claras e objetivas que apresentem diretamente as respostas às perguntas do negócio.

◆ Apoie com Visualizações

Sustente cada conclusão construída utilizando os gráficos e elementos visuais gerados no projeto.

◆ Destaque Padrões Surpreendentes

Evidencie as conexões mais inesperadas ou relevantes encontradas durante a Análise Exploratória.



Preparação Executiva

Apresentar para a turma capacita o estudante a traduzir jargões técnicos em insights de negócios compreensíveis, preparando-os de forma ideal para o dinâmico ambiente corporativo.

Construindo seu Portfólio de Ciência de Dados

1. Opções de Exportação (Google Colab)

O Google Colab permite exportar seus notebooks finalizados em formatos ideais para entrega e compartilhamento:



Formato .ipynb

Notebook interativo completo com código e saídas salvas.



Formato PDF

Relatório estático ideal para apresentações e leituras rápidas.

Construir um portfólio profissional é vital para ingressar no mercado de trabalho. Um portfólio bem documentado, mostrando código limpo e conclusões lógicas, demonstra competência técnica e capacidade analítica para recrutadores de Ciência de Dados.

2. Plataformas de Hospedagem Ideais

Recomenda-se publicar e documentar sistematicamente suas análises exploratórias em repositórios e comunidades consolidadas:

◆ Repositórios do GitHub

O principal hub de código do mundo. Organize seus projetos com arquivos README.md detalhados para explicar seu raciocínio, metodologias utilizadas e principais insights obtidos.

◆ Plataforma do Kaggle

Compartilhe seus notebooks diretamente onde a comunidade de ciência de dados está ativa. Excelente para demonstrar participação em competições e uso de conjuntos de dados públicos.



Dica de Sucesso: Sempre limpe os comentários temporários e garanta que todas as células do notebook rodem sem erros de cima para baixo antes de exportar.

Estudos Continuados: Recursos Indispensáveis

Guia de Recursos Recomendados

■ Documentação Oficial do Pandas

A fonte definitiva para dominar a sintaxe correta e encontrar métodos detalhados de manipulação de dados.

■ Comunidade StackOverflow

O fórum principal global para tirar dúvidas técnicas, resolver erros de código e contornar bugs específicos.

■ Notebooks Públicos do Kaggle

Explore milhares de códigos de cientistas de dados experientes para aprender novas técnicas avançadas de análise e visualização gráfica.

Perguntas Frequentes (FAQ)

Como proceder ao encontrar um bug complexo?

Copie a mensagem de erro exata e consulte o StackOverflow. Quase todos os erros iniciais comuns já foram resolvidos e documentados por lá.

Por que analisar notebooks públicos ajuda tanto?

No Kaggle, você vê na prática como profissionais estruturam pipelines de dados. Isso ajuda a construir repertório técnico e boas práticas visuais.

Próxima Aula: Avançando para Manipulação Avançada

Plano de Lição de Casa



Análise Univariada

Finalizar a análise univariada do dataset escolhido hoje como preparação básica.



Elaboração de Perguntas

Trazer pelo menos três perguntas complexas que exijam o cruzamento de duas ou mais variáveis para serem respondidas.

Aula 2: Transição de Conceitos

Deixaremos a fase de exploração inicial para dominar manipulações complexas de dados.

Tratamento de Dados Nulos

Agrupamentos com `groupby()`

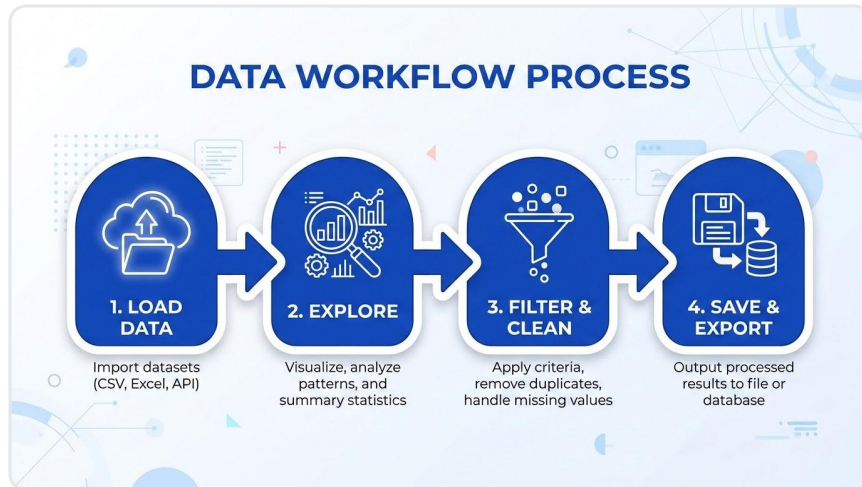
Junção de Tabelas (Merges & Joins)

Resumo da Aula 1

Competências Conquistadas Hoje:

- ✓ Python serve como base para cálculos matemáticos.
- ✓ O Pandas é a biblioteca padrão para estruturas tabulares (DataFrames).
- ✓ Importação de dados via `pd.read_csv()`.
- ✓ Métodos de exploração inicial: `head()`, `info()`, `describe()`.
- ✓ Seleção de colunas e filtragem de linhas com indexação booleana.
- ✓ Criação de visualizações básicas (Histogramas).
- ✓ Uso do Kaggle para encontrar datasets reais.

Caminho Analítico Simplificado:



Nota do Processo:

Esta esteira representa o ciclo essencial de qualquer projeto de dados: carregar o arquivo, explorar sua estrutura, refinar o escopo e exportar os resultados consolidados para modelagem.

Obrigado e até a próxima aula!

Chegamos ao fim da nossa introdução à Ciência de Dados com Python. O domínio dessas ferramentas requer prática contínua e curiosidade analítica.

Continuem explorando dados e testando novos códigos no Google Colab. Obrigado pela participação e até a próxima sessão prática.

Suporte e Dúvidas

Para suporte, dúvidas ou feedback sobre a aula, utilizem sempre os canais oficiais.

Canais de Comunicação

Acesse a disciplina na plataforma acadêmica ou participe ativamente no fórum da turma.

“A prática leva à perfeição”